

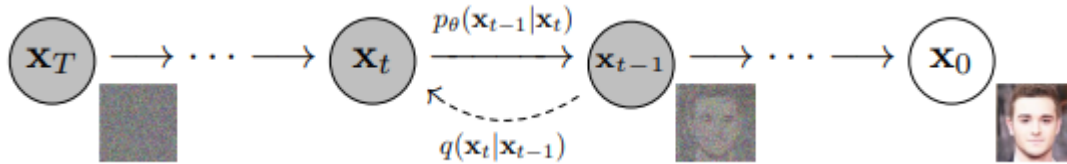
# CVPDL hw3 Image Generation Report

School：國立清華大學 National Tsing Hua University

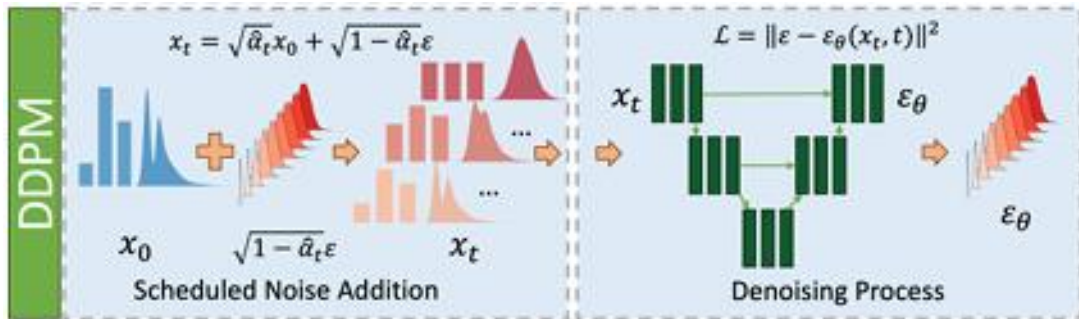
Student ID：114064545

Name：謝尚哲

## 1. Model Description



▲ 圖 1, Directed graphical model of DDPM [1]



▲ 圖 2, DDPM 架構圖[2]，圖中 Scheduled Noise Addition 即為 Forward Diffusion；Denoising Process 即為 Reverse Diffusion

本次作業我是以 DDPM [1]的 Diffusion graphical model (圖 1)為基本架構，採用和 DDPM 相同的 diffusion 概念以及數學架構進行簡化，以實現 Forward Diffusion (1)

以及 Reverse Diffusion (2)。

其中 **Forward Diffusion** 公式為：

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I}) \quad (1)$$

將乾淨的影像  $\mathbf{x}_0$  透過 beta schedule ( $\beta_t := 1 - \alpha_t$  且  $\beta_t$  從  $1e-4$  隨著  $t$  增加至  $0.02$ ) 來控制每一步加入多少 Gaussian Noise (epsilon) 使得 MNIST 數字圖能夠隨著 timestep 增加，在  $t=T$  時得到  $\mathbf{x}_{t=T} \approx N(0, I)$ ，過程如圖 2 所示。

另外 **Reverse Diffusion** 公式為：

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) := \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \boldsymbol{\Sigma}_\theta(\mathbf{x}_t, t)) \quad (2)$$

輸入為  $x_t$  並於 Feature Maps 加入 Sinusoidal Position Embedding，使用 UNet 的 Down sampling 和 Up sampling 架構得到輸出 Predicted Noise  $\epsilon_\theta$ ，最後再利用 (2) 求出  $x_{t-1}$ 。由 pure noise 一步步去噪，最終還原出類似 MNIST 的影像。

此外，針對這次的資料集為大小 28\*28 的 MNIST 數字集，不需要依照 DDPM 規格的架構訓練，因此我捨棄使用 Attention，並且大幅簡化原本針對 UNet 的設計—Down sampling 及 Up sampling 皆只有兩層。

## 2. Implementation Details

- **Dataset**

```
mnist/
├── train/
│   ├── 00001.png
│   ├── ...
│   └── 60000.png
```

- **Data Preprocessing**

- **Resize 至大小 28x28:** 確保和簡化版 UNet (即 code 中 UNetSmall)輸入維度相同。
- **ToTensor:** 將 RGB 的圖像轉成 Tensor，並且縮放至[0, 1]
- **Normalize:** 使資料從[-1, 1]改成落在[0, 1]
- **Data Loader 相關設定**—shuffle, num\_workers etc.

- **Training Strategies**

- **使用 DDPM 為基礎數學架構：**此篇論文是針對 Diffusion Model 問題非常經典之作，且我認為本次 MNIST 數字集適合使用 DDPM 概念解決。
- **簡化 UNet 架構：**

| UNet of DDPM[1]           | UNetSmall                                                |
|---------------------------|----------------------------------------------------------|
| Down sampling: $\geq 3$ 層 | Down sampling: 2 層 (28 $\rightarrow$ 14 $\rightarrow$ 7) |
| Up sampling: $\geq 3$ 層   | Up sampling: 2 層 (7 $\rightarrow$ 14 $\rightarrow$ 28)   |
| 多層 Self-Attention         | 未使用 Attention                                            |

- **Loss Function：**和 DDPM 的 Simplified Loss Function 相符—公式如 (3)，同圖 2 的 Loss Function，模型僅需預測在 timestep t 中加入的雜訊  $\epsilon$

$$L_{simple} = E[\|\epsilon - \epsilon_\theta(xt, t)\|^2] \quad (3)$$

- **Beta Scheduling:** 和 DDPM 相符 — 從  $1e-4$  隨著 timesteps  $t$  線性增加至 0.02，且  $T = 1000$  steps，以確保 Forward Diffusion 能夠更加穩定。

### 3. Result Analysis

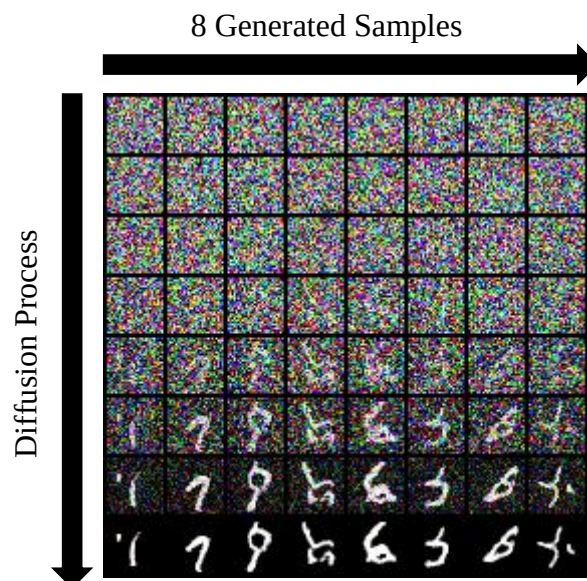
- **Quantitative Improvements**

\*base 為 UNet 的 feature channels，也就是模型容量，調越高代表模型越加複雜。當調到 base=192 時，在 50 epochs 且 learning rate= $2e-4$  的情況下，loss 會暴增造成不可逆的結果（在本次實驗是如此），因此只要 base=192 時一律降低至 35 epochs 訓練以維持模型穩定。

# of attempts: 9 times

| Settings (w/o pretrained weight and external datasets)                                                                                    | FID          |
|-------------------------------------------------------------------------------------------------------------------------------------------|--------------|
| epochs = 50, lr = $1e-4$ , base = 64, beta = linear( $1e-4 \rightarrow 0.02$ )                                                            | 189.69       |
| epochs = 50, <b>lr = <math>2e-4</math></b> , base = 64, beta = linear( $1e-4 \rightarrow 0.02$ )                                          | 122.97       |
| epochs = 50, lr = $2e-4$ , <b>base = 128</b> , beta = linear( $1e-4 \rightarrow 0.02$ )                                                   | 61.61        |
| <b>epochs = 35</b> , lr = $2e-4$ , <b>base = 192*</b> , beta = linear( $1e-4 \rightarrow 0.02$ )                                          | 52.04        |
| epochs = 35, lr = $2e-4$ , <b>base = 128</b> , beta = linear( $1e-4 \rightarrow 0.02$ )<br>+ 1 layer Attention                            | 72.48        |
| <b>epochs = 50</b> , lr = $2e-4$ , base = 128, beta = linear( $1e-4 \rightarrow 0.02$ )<br>+1 layer Attention + <b>Use best inference</b> | 54.06        |
| <b>epochs = 35</b> , lr = $2e-4$ , <b>base = 192</b> , beta = linear( $1e-4 \rightarrow 0.02$ )<br>+ Use best inference                   | 45.99        |
| <b>epochs = 80</b> , <b>lr = <math>1e-4</math></b> , base = 192, beta = linear( $1e-4 \rightarrow 0.02$ )<br>+Use best inference          | 35.24        |
| <b>epochs=150</b> , <b>lr = <math>8e-5</math></b> , <b>base=128</b> , beta = linear( $1e-4 \rightarrow 0.02$ )<br>+Use best inference     | <b>25.53</b> |

- **Diffusion Process Visualizations**



## 4. Short Conclusion

- 由上圖 Diffusion Process Visualization 可以得知模型具備逐步去噪能力，可從純高斯雜訊還原至部分合理的數字形狀。
- 由訓練結果可知，儘管模型架構被大幅簡化，DDPM 仍能在低解析度影像上產生穩定且高品質的生成結果。

## 5. Reference

- [1] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2020
- [2] T. Sattarov, M. Schreyer, and D. Borth, “Diffusion-Scheduled Denoising Autoencoders for Anomaly Detection in Tabular Data,” in *Proc. 31st ACM SIGKDD Conf. Knowledge Discovery and Data Mining (KDD '25)*, Toronto, ON, Canada, 2025